JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DatagramChannel

java.lang.Object
  java.nio.channels.spi.AbstractInterruptibleChannel
    java.nio.channels.SelectableChannel
      java.nio.channels.spi.AbstractSelectableChannel
        java.nio.channels.DatagramChannel

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `ByteChannel`, `Channel`, `GatheringByteChannel`, `InterruptibleChannel`, `MulticastChannel`, `NetworkChannel`, `ReadableByteChannel`, `ScatteringByteChannel`, `WritableByteChannel`

```
public abstract class DatagramChannel
extends AbstractSelectableChannel
implements ByteChannel, ScatteringByteChannel, GatheringByteChannel,
MulticastChannel
```

A selectable channel for datagram-oriented sockets.

A datagram channel is created by invoking one of the `open` methods of this class. It is not possible to create a channel for an arbitrary, pre-existing datagram socket. A newly-created datagram channel is open but not connected. A datagram channel need not be connected in order for the `send` and `receive` methods to be used. A datagram channel may be connected, by invoking its `connect` method, in order to avoid the overhead of checks performed as part of every send and receive operation. A datagram channel must be connected in order to use the `read` and `write` methods, since those methods do not accept or return socket addresses.

Once connected, a datagram channel remains connected until it is disconnected or closed. Whether or not a datagram channel is connected may be determined by invoking its `isConnected` method.

Socket options are configured using the `setOption` method. A datagram channel to an Internet Protocol socket supports the following options:

Option Name	Description
`SO_SNDBUF`	The size of the socket send buffer
`SO_RCVBUF`	The size of the socket receive buffer
`SO_REUSEADDR`	Re-use address
`SO_BROADCAST`	Allow transmission of broadcast datagrams
`IP_TOS`	The Type of Service (ToS) octet in the Internet Protocol (IP) header
`IP_MULTICAST_IF`	

Option Name	Description
	The network interface for Internet Protocol (IP) multicast datagrams
IP_MULTICAST_TTL	The *time-to-live* for Internet Protocol (IP) multicast datagrams
IP_MULTICAST_LOOP	Loopback for Internet Protocol (IP) multicast datagrams

Additional (implementation specific) options may also be supported.

Datagram channels are safe for use by multiple concurrent threads. They support concurrent reading and writing, though at most one thread may be reading and at most one thread may be writing at any given time.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**DatagramChannel**(SelectorProvider provid	Initializes a new instance of this class.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
abstract DatagramChannel	**bind**(SocketAddress local)	Binds the channel's socket to a local address.
abstract DatagramChannel	**connect**(SocketAddress remote)	Connects this channel's socket.
abstract DatagramChannel	**disconnect**()	Disconnects this channel's socket.
abstract SocketAddress	**getLocalAddress**()	Returns the socket address that this channel's socket is bound to.
	getRemoteAddress()	

abstract SocketAddress		Returns the remote address to which this channel's socket is connected.
abstract boolean	**isConnected**()	Tells whether or not this channel's socket is connected.
static DatagramChannel	**open**()	Opens a datagram channel.
static DatagramChannel	**open**(ProtocolFamily family)	Opens a datagram channel.
abstract int	**read**(ByteBuffer dst)	Reads a datagram from this channel.
final long	**read**(ByteBuffer[] dsts)	Reads a datagram from this channel.
abstract long	**read**(ByteBuffer[] dsts, int offset, int length)	Reads a datagram from this channel.
abstract SocketAddress	**receive**(ByteBuffer dst)	Receives a datagram via this channel.
abstract int	**send**(ByteBuffer src, SocketAddress target)	Sends a datagram via this channel.
abstract <T> DatagramChannel	**setOption**(SocketOption<T> name, T value)	Sets the value of a socket option.
abstract DatagramSocket	**socket**()	Retrieves a datagram socket associated with this channel.
final int	**validOps**()	Returns an operation set identifying this channel's supported operations.
abstract int	**write**(ByteBuffer src)	Writes a datagram to this channel.
final long	**write**(ByteBuffer[] srcs)	Writes a datagram to this channel.

abstract long	**write**(ByteBuffer[] srcs, int offset, int length)	Writes a datagram to this channel.

Methods declared in class AbstractSelectableChannel

blockingLock, configureBlocking, implCloseChannel, implCloseSelectableChannel, implConfigureBlocking, isBlocking, isRegistered, keyFor, provider, register

Methods declared in class SelectableChannel

register

Methods declared in class AbstractInterruptibleChannel

begin, close, end, isOpen

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods declared in interface MulticastChannel

close, join, join

Methods declared in interface NetworkChannel

getOption, supportedOptions

Constructor Details

DatagramChannel

```
protected DatagramChannel(SelectorProvider provider)
```

Initializes a new instance of this class.

Parameters:

`provider` - The provider that created this channel

Method Details

open

```
public static DatagramChannel open()
                              throws IOException
```

Opens a datagram channel.

The new channel is created by invoking the `openDatagramChannel` method of the system-wide default `SelectorProvider` object. The channel will not be connected.

The `ProtocolFamily` of the channel's socket is platform (and possibly configuration) dependent and therefore unspecified. The `open` allows the protocol family to be selected when opening a datagram channel, and should be used to open datagram channels that are intended for Internet Protocol multicasting.

Returns:

A new datagram channel

Throws:

`IOException` - If an I/O error occurs

See Also:

java.net.preferIPv4Stack system property

open

```
public static DatagramChannel open(ProtocolFamily family)
                              throws IOException
```

Opens a datagram channel.

The `family` parameter is used to specify the `ProtocolFamily`. If the datagram channel is to be used for IP multicasting then this should correspond to the address type of the multicast groups that this channel will join.

The new channel is created by invoking the `openDatagramChannel` method of the system-wide default `SelectorProvider` object. The channel will not be connected.

API Note:

Unix domain sockets are not supported.

Parameters:

`family` - The protocol family

Returns:

A new datagram channel

Throws:

`UnsupportedOperationException` - If the specified protocol family is not supported. For example, suppose the parameter is specified as `StandardProtocolFamily.INET6` but IPv6 is not enabled on the platform.

`IOException` - If an I/O error occurs

Since:

1.7

See Also:

java.net.preferIPv4Stack system property

validOps

```
public final int validOps()
```

Returns an operation set identifying this channel's supported operations.

Datagram channels support reading and writing, so this method returns (`SelectionKey.OP_READ` | `SelectionKey.OP_WRITE`).

Specified by:

`validOps` in class `SelectableChannel`

Returns:

The valid-operation set

bind

```
public abstract DatagramChannel bind(SocketAddress local)
                              throws IOException
```

Description copied from interface: `NetworkChannel`

Binds the channel's socket to a local address.

This method is used to establish an association between the socket and a local address. Once an association is established then the socket remains bound until the channel is closed. If the `local` parameter has the value `null` then the socket will be bound to an address that is assigned automatically.

Specified by:

`bind` in interface `NetworkChannel`

Parameters:

`local` - The address to bind the socket, or `null` to bind the socket to an automatically assigned socket address

Returns:

This channel

Throws:

`AlreadyBoundException` - If the socket is already bound

`UnsupportedAddressTypeException` - If the type of the given address is not supported

`ClosedChannelException` - If the channel is closed

`IOException` - If some other I/O error occurs

Since:

1.7

See Also:

`NetworkChannel.getLocalAddress()`

setOption

```
public abstract <T> DatagramChannel setOption(SocketOption<T> name,
                                              T value)
                                       throws IOException
```

Description copied from interface: `NetworkChannel`

Sets the value of a socket option.

Specified by:

`setOption` in interface `NetworkChannel`

Type Parameters:

`T` - The type of the socket option value

Parameters:

`name` - The socket option

`value` - The value of the socket option. A value of `null` may be a valid value for some socket options.

Returns:

This channel

Throws:

`UnsupportedOperationException` - If the socket option is not supported by this channel

`IllegalArgumentException` - If the value is not a valid value for this socket option

`ClosedChannelException` - If this channel is closed

`IOException` - If an I/O error occurs

Since:

1.7

See Also:

`StandardSocketOptions`

socket

```
public abstract DatagramSocket socket()
```

Retrieves a datagram socket associated with this channel.

Returns:

A datagram socket associated with this channel

isConnected

```
public abstract boolean isConnected()
```

Tells whether or not this channel's socket is connected.

Returns:

`true` if, and only if, this channel's socket is `open` and connected

connect

```
public abstract DatagramChannel connect(SocketAddress remote)
                                 throws IOException
```

Connects this channel's socket.

The channel's socket is configured so that it only receives datagrams from, and sends datagrams to, the given remote *peer* address. Once connected, datagrams may not be received from or sent to any other address. Datagrams in the channel's socket receive buffer, which have not been received before invoking this method, may be discarded. The channel's socket remains connected until it is explicitly disconnected or until it is closed.

This method may be invoked at any time. If another thread has already initiated a read or write operation upon this channel, then an invocation of this method will block until any such operation is complete. If this channel's socket is not bound then this method will first cause the socket to be bound to an address that is assigned automatically, as if invoking the `bind` method with a parameter of `null`.

Parameters:

`remote` - The remote address to which this channel is to be connected

Returns:

This datagram channel

Throws:

`AlreadyConnectedException` - If this channel is already connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the connect operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the connect operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`UnresolvedAddressException` - If the given remote address is not fully resolved

`UnsupportedAddressTypeException` - If the type of the given remote address is not supported

`IOException` - If some other I/O error occurs

disconnect

```
public abstract DatagramChannel disconnect()
                                        throws IOException
```

Disconnects this channel's socket.

The channel's socket is configured so that it can receive datagrams from, and sends datagrams to, any remote address.

This method may be invoked at any time. If another thread has already initiated a read or write operation upon this channel, then an invocation of this method will block until any such operation is complete.

If this channel's socket is not connected, or if the channel is closed, then invoking this method has no effect.

API Note:

If this method throws an IOException, the channel's socket may be left in an unspecified state. It is strongly recommended that the channel be closed when disconnect fails.

Returns:

This datagram channel

Throws:

`IOException` - If some other I/O error occurs

getRemoteAddress

```
public abstract SocketAddress getRemoteAddress()
                                          throws IOException
```

Returns the remote address to which this channel's socket is connected.

Returns:

The remote address; `null` if the channel's socket is not connected

Throws:

`ClosedChannelException` - If the channel is closed

`IOException` - If an I/O error occurs

Since:

1.7

receive

```
public abstract SocketAddress receive(ByteBuffer dst)
                               throws IOException
```

Receives a datagram via this channel.

If a datagram is immediately available, or if this channel is in blocking mode and one eventually becomes available, then the datagram is copied into the given byte buffer and its source address is returned. If this channel is in non-blocking mode and a datagram is not immediately available then this method immediately returns `null`.

The datagram is transferred into the given byte buffer starting at its current position, as if by a regular `read` operation. If there are fewer bytes remaining in the buffer than are required to hold the datagram then the remainder of the datagram is silently discarded.

This method may be invoked at any time. If another thread has already initiated a read operation upon this channel, however, then an invocation of this method will block until the first operation is complete. If this channel's socket is not bound then this method will first cause the socket to be bound to an address that is assigned automatically, as if invoking the `bind` method with a parameter of `null`.

Parameters:

`dst` - The buffer into which the datagram is to be transferred

Returns:

The datagram's source address, or `null` if this channel is in non-blocking mode and no datagram was immediately available

Throws:

`IllegalArgumentException` - If the buffer is read-only

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the read operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

send

```
public abstract int send(ByteBuffer src,
                         SocketAddress target)
                  throws IOException
```

Sends a datagram via this channel.

If this channel is in non-blocking mode and there is sufficient room in the underlying output buffer, or if this channel is in blocking mode and sufficient room becomes available, then the remaining bytes in the given buffer are transmitted as a single datagram to the given target address.

The datagram is transferred from the byte buffer as if by a regular `write` operation.

This method may be invoked at any time. If another thread has already initiated a write operation upon this channel, however, then an invocation of this method will block until the first operation is complete. If this channel's socket is not bound then this method will first cause the socket to be bound to an address that is assigned automatically, as if by invoking the `bind` method with a parameter of `null`.

Parameters:

`src` - The buffer containing the datagram to be sent

`target` - The address to which the datagram is to be sent

Returns:

The number of bytes sent, which will be either the number of bytes that were remaining in the source buffer when this method was invoked or, if this channel is non-blocking, may be zero if there was insufficient room for the datagram in the underlying output buffer

Throws:

`AlreadyConnectedException` - If this channel is connected to a different address from that specified by `target`

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the read operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`UnresolvedAddressException` - If the given remote address is not fully resolved

`UnsupportedAddressTypeException` - If the type of the given remote address is not supported

`IOException` - If some other I/O error occurs

read

```
public abstract int read(ByteBuffer dst)
                  throws IOException
```

Reads a datagram from this channel.

This method may only be invoked if this channel's socket is connected, and it only accepts datagrams from the socket's peer. If there are more bytes in the datagram than remain in the given buffer then the remainder of the datagram is silently discarded. Otherwise this method behaves exactly as specified in the `ReadableByteChannel` interface.

Specified by:

`read` in interface `ReadableByteChannel`

Parameters:

`dst` - The buffer into which bytes are to be transferred

Returns:

The number of bytes read, possibly zero, or `-1` if the channel has reached end-of-stream

Throws:

`NotYetConnectedException` - If this channel's socket is not connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the read operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

IOException - If some other I/O error occurs

read

```
public abstract long read(ByteBuffer[] dsts,
                          int offset,
                          int length)
                   throws IOException
```

Reads a datagram from this channel.

This method may only be invoked if this channel's socket is connected, and it only accepts datagrams from the socket's peer. If there are more bytes in the datagram than remain in the given buffers then the remainder of the datagram is silently discarded. Otherwise this method behaves exactly as specified in the ScatteringByteChannel interface.

Specified by:

read in interface ScatteringByteChannel

Parameters:

dsts - The buffers into which bytes are to be transferred

offset - The offset within the buffer array of the first buffer into which bytes are to be transferred; must be non-negative and no larger than dsts.length

length - The maximum number of buffers to be accessed; must be non-negative and no larger than dsts.length - offset

Returns:

The number of bytes read, possibly zero, or -1 if the channel has reached end-of-stream

Throws:

NotYetConnectedException - If this channel's socket is not connected

ClosedChannelException - If this channel is closed

AsynchronousCloseException - If another thread closes this channel while the read operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

IOException - If some other I/O error occurs

read

```
public final long read(ByteBuffer[] dsts)
                throws IOException
```

Reads a datagram from this channel.

This method may only be invoked if this channel's socket is connected, and it only accepts datagrams from the socket's peer. If there are more bytes in the datagram than remain in the given buffers then the remainder of the datagram is silently discarded. Otherwise this method behaves exactly as specified in the `ScatteringByteChannel` interface.

Specified by:

`read` in interface `ScatteringByteChannel`

Parameters:

`dsts` - The buffers into which bytes are to be transferred

Returns:

The number of bytes read, possibly zero, or `-1` if the channel has reached end-of-stream

Throws:

`NotYetConnectedException` - If this channel's socket is not connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the read operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

write

```
public abstract int write(ByteBuffer src)
                  throws IOException
```

Writes a datagram to this channel.

This method may only be invoked if this channel's socket is connected, in which case it sends datagrams directly to the socket's peer. Otherwise it behaves exactly as specified in the `WritableByteChannel` interface.

Specified by:

`write` in interface `WritableByteChannel`

Parameters:

`src` - The buffer from which bytes are to be retrieved

Returns:

The number of bytes written, possibly zero

Throws:

`NotYetConnectedException` - If this channel's socket is not connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the write operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

write

```
public abstract long write(ByteBuffer[] srcs,
                           int offset,
                           int length)
                    throws IOException
```

Writes a datagram to this channel.

This method may only be invoked if this channel's socket is connected, in which case it sends datagrams directly to the socket's peer. Otherwise it behaves exactly as specified in the `GatheringByteChannel` interface.

Specified by:

`write` in interface `GatheringByteChannel`

Parameters:

`srcs` - The buffers from which bytes are to be retrieved

`offset` - The offset within the buffer array of the first buffer from which bytes are to be retrieved; must be non-negative and no larger than `srcs.length`

`length` - The maximum number of buffers to be accessed; must be non-negative and no larger than `srcs.length` - `offset`

Returns:

The number of bytes sent, which will be either the number of bytes that were remaining in the source buffer when this method was invoked or, if this channel is non-blocking, may be zero if there was insufficient room for the datagram in the underlying output buffer

Throws:

`NotYetConnectedException` - If this channel's socket is not connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the write operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

write

```
public final long write(ByteBuffer[] srcs)
                 throws IOException
```

Writes a datagram to this channel.

This method may only be invoked if this channel's socket is connected, in which case it sends datagrams directly to the socket's peer. Otherwise it behaves exactly as specified in the `GatheringByteChannel` interface.

Specified by:

`write` in interface `GatheringByteChannel`

Parameters:

`srcs` - The buffers from which bytes are to be retrieved

Returns:

The number of bytes sent, which will be either the number of bytes that were remaining in the source buffer when this method was invoked or, if this channel is non-blocking, may be zero if there was insufficient room for the datagram in the underlying output buffer

Throws:

`NotYetConnectedException` - If this channel's socket is not connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the write operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

getLocalAddress

```
public abstract SocketAddress getLocalAddress()
                                        throws IOException
```

Returns the socket address that this channel's socket is bound to.

Where the channel is `bound` to an Internet Protocol socket address then the return value from this method is of type `InetSocketAddress`.

If the channel's socket was initially bound to the wildcard address and is now `connected`, then the address returned may be the local address selected as the source address for datagrams sent via this channel instead of the wildcard address. When `disconnect()` is called, the bound address reverts to the wildcard address.

Specified by:

`getLocalAddress` in interface `NetworkChannel`

Returns:

The `SocketAddress` that the socket is bound to; `null` if the channel's socket is not bound

Throws:

`ClosedChannelException` - If the channel is closed

`IOException` - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.